

Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



A Project Proposal

Real-Time Sign Language Recognition Mobile Application (Sign-to-Text)

[Code No: COMP 311]

(For partial fulfillment of III Year/ II Semester in Computer Science)

Submitted by :
Arnav Bhatta(05)
Yunika Bhochhibhoya(07)
Anmol Dahal(11)
Ranjita Dhakal(12)
Innima Karki(16)

Submitted to:

Mr. Bal Krishna Bal Sir
Department of Computer Science and Engineering

Submission Date: 09-11-2025

Date: 9th November, 2025

To,

The Project Supervisor,

Department of Computer Science and Engineering,

Kathmandu University

Subject: Cover Letter for Proposal Approval

Dear Sir/Madam,

We are submitting our proposal for this semester's project, '**Sign-to-Text**', a real-time, camera-based mobile application aimed at improving communication for individuals with hearing and speech impairments

The system will use **MediaPipe Hand Tracking** to extract 21 key hand landmarks, while a **machine learning model** trained on these features will classify gestures into corresponding alphabets or words. To ensure efficiency and accessibility, the model will be optimized with **TensorFlow Lite**, allowing lightweight, offline performance on mobile devices. Recognized text will be displayed instantly, enabling smooth and practical interaction.

Our proposal outlines the **background, objectives, methodology, and expected outcomes**, with a detailed structure provided in the Table of Contents. We sincerely value your guidance and feedback as we move forward with this project.

Sincerely,

Arnav Bhatta(05)

Yunika Bhochhibhoya(07)

Anmol Dahal(11)

Ranjita Dhakal(12)

Innima Karki(16)

Abstract

Communication is a fundamental aspect of human interaction, and there are several methods to convey information. Verbal communication is one of the most widely used methods; however, individuals with hearing and speech impairments often face challenges in expressing themselves. Sign language bridges this gap but is not widely understood by the general population, limiting effective communication.

This project proposes a real-time, camera-based mobile application for sign language recognition and conversion to text (Sign-to-Text). The application captures hand gestures through the mobile device's camera and employs MediaPipe Hand Tracking to extract 21 key hand landmarks. A machine learning model, trained on these landmarks, classifies gestures into corresponding letters or words. The system is optimized for mobile deployment using TensorFlow Lite, ensuring lightweight and efficient performance without requiring an internet connection. Recognized text is displayed instantly on-screen, enabling immediate communication.

This project aims to provide a practical, user-friendly, and inclusive solution to support individuals with hearing and speech impairments in everyday interactions.

Keywords: *Sign Language Recognition, Hand Gesture Recognition, MediaPipe Hand Tracking, Machine Learning, Inclusive Communication*

Table Of Contents

List of Figures	5
Acronyms/Abbreviations	6
Chapter 1: Introduction	7
1.1 Background	7
1.2 Objectives	8
1.3 Motivation and Significance	8
1.4 Expected Outcomes	9
Chapter 2: Related works	10
Chapter 3: Procedures and Methods	13
3.1. Data Collection	13
3.2. Feature Extraction	13
3.3. Model Development	13
3.4. Mobile Optimization and Deployment	13
3.5. Real-Time Gesture Recognition	14
3.6. Testing and Evaluation	14
Chapter 4: System Requirement Specifications	16
b. Backend	16
Chapter 5: Project Planning and Scheduling	18
Appendix	19
References	20

List of Figures

Fig. No.	Title	Page
Fig 1	ASL Sign Language Pocket Sign Logo	11
Fig 2	Lingvano App Screenshot	12
Fig 3	ASL Bloom – Sign Language App Screenshot	12
Fig 4	Use-Case Diagram for Sign-to-Text Application	15
Fig 5	Gantt Chart – Project Planning and Scheduling	18

Acronyms/Abbreviations

Acronym	Full Form
ASL	American Sign Language
API	Application Programming Interface
CNN	Convolutional Neural Network
DRF	Django Rest Framework
FPS	Frames Per Second
JWT	JSON Web Token
ML	Machine Learning
TFLite	TensorFlow Lite
UI	User Interface

Chapter 1: Introduction

1.1 Background

Communication is one of the most essential aspects of human interaction, enabling people to exchange ideas, express emotions, and build relationships. While verbal communication is the most common method, individuals with hearing and speech impairments often encounter significant barriers in expressing themselves effectively. For these individuals, sign language serves as a vital mode of communication. However, because sign language is not widely understood by the general population, its effectiveness remains limited in everyday interactions.

The gap between sign language users and non-signers creates challenges in social, educational, and professional environments. In recent years, advancements in computer vision and machine learning have opened new possibilities for developing intelligent systems that can bridge this communication divide. By leveraging real-time gesture recognition, it is possible to translate sign language into text, making interactions more inclusive and accessible.

This project proposes the development of a real-time, camera-based mobile application for sign language recognition and conversion to text (Sign-to-Text). The system utilizes MediaPipe Hand Tracking to extract 21 key hand landmarks, which are then classified into corresponding letters or words using a machine learning model. To ensure efficiency and portability, the model is optimized with TensorFlow Lite, allowing the application to run smoothly on mobile devices without requiring internet connectivity. Recognized text is displayed instantly on-screen, enabling immediate and practical communication.

By combining accessibility, efficiency, and ease of use, this project aims to provide a user-friendly and inclusive solution that empowers individuals with hearing and speech impairments in their daily interactions, while also raising awareness and fostering inclusivity within the broader community.

1.2 Objectives

- To develop a real-time mobile application for sign language recognition and conversion to text.
- To use MediaPipe Hand Tracking for extracting 21 hand landmarks from gestures.
- To train a machine learning model for accurate gesture classification.
- To optimize the model with TensorFlow Lite for lightweight and offline performance.
- To design a user-friendly interface that displays recognized text instantly.

1.3 Motivation and Significance

Individuals with hearing and speech impairments often face challenges in communicating effectively, as sign language is not widely understood by the general population. This creates barriers in social, educational, and professional settings, limiting inclusion and equal opportunities. The motivation for this project is to leverage advances in computer vision and machine learning to develop a practical, real-time solution that can bridge this communication gap. By using MediaPipe Hand Tracking and a TensorFlow Lite optimized model, the system provides accurate gesture recognition and instant text conversion on mobile devices, even offline. This project is significant because it enhances accessibility, fosters inclusivity, and offers a lightweight, user-friendly tool that empowers individuals with hearing and speech impairments to communicate independently in everyday interactions.

1.4 Expected Outcomes

This project aims to develop a real-time, mobile-based sign language recognition application that is both user-friendly and efficient. The system will accurately capture hand gestures through the mobile device's camera and employ MediaPipe Hand Tracking to extract 21 key hand landmarks. A machine learning model trained on these features will classify gestures into corresponding letters or words, with results displayed instantly as text on the screen. To ensure smooth performance on mobile devices, the model will be optimized using TensorFlow Lite, enabling offline functionality without requiring an internet connection.

The application will provide a practical and accessible tool for individuals with hearing and speech impairments, facilitating communication in everyday situations. Additionally, comprehensive documentation covering system design, gesture recognition workflow, model training, and user guidelines will be provided. Through this project, the expected outcome is a reliable, inclusive, and efficient communication platform that enhances accessibility while offering valuable learning in mobile application development, computer vision, and machine learning deployment.

Chapter 2: Related works

Sign language recognition has been an active area of research, combining computer vision, machine learning, and mobile technology to bridge communication gaps for individuals with hearing and speech impairments.

Below are a few works that have existed that have similar objectives to ours.

1. ASL Sign Language Pocket Sign

An Android/IOS app featuring hundreds of video lessons for learning ASL. It includes interactive quizzes and a dictionary for effective learning.



Fig 1: ASL Sign Language Pocket Sign Logo

1. Lingvano-Learn Sign Language

Lingvano is a comprehensive mobile application designed to facilitate the learning of American Sign Language (ASL). It offers a structured curriculum suitable for beginners, featuring over 600 video lessons created by Deaf instructors. These lessons encompass vocabulary, grammar, and real-life dialogues, enabling users to engage in basic conversations with just 10 minutes of daily practice

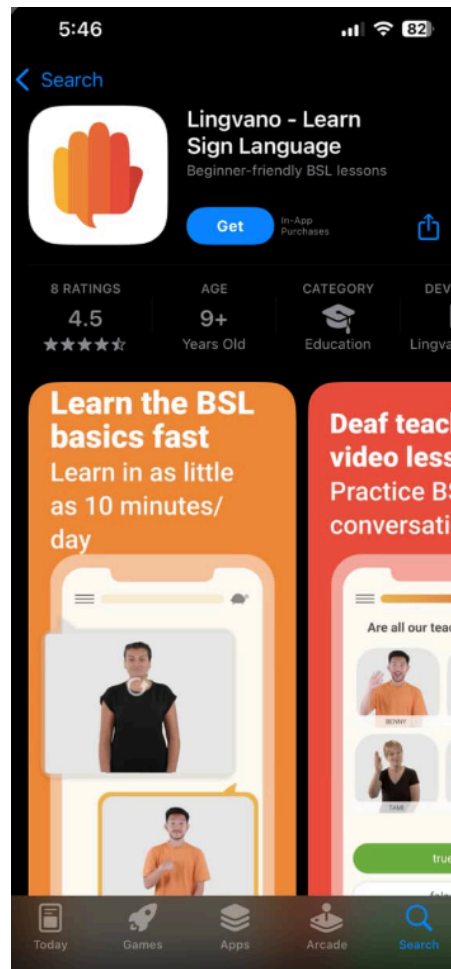


Fig 2: Lingvano App

2. ASL Bloom

ASL Bloom is a mobile app available on Android and iOS that helps users learn American Sign Language through gamified lessons and quizzes. It offers over 1,300 signs and sentences across multiple modules, includes a visual dictionary, and provides interactive exercises to reinforce learning. The app is designed to make ASL learning engaging and effective for beginners and intermediate users.

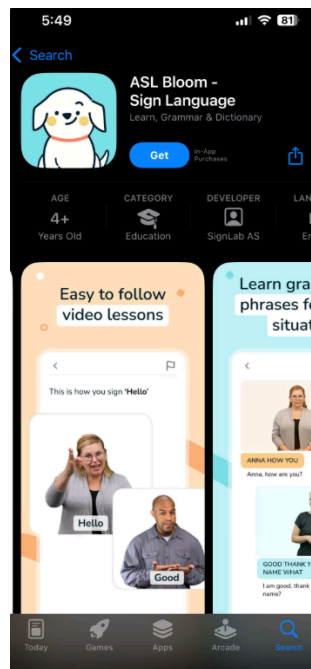


Fig 3: ASL Bloom - Sign Language

Chapter 3: Procedures and Methods

3.1. Data Collection

- Gather a dataset of hand gestures corresponding to the alphabet, numbers, and commonly used words in sign language.
- Capture images and video sequences under various lighting and background conditions to improve model robustness.
- Label the dataset accurately to ensure correct gesture-to-text mapping.

3.2. Feature Extraction

- Use MediaPipe Hand Tracking to extract 21 key hand landmarks from each gesture.
- Normalize landmark coordinates to handle variations in hand size, orientation, and distance from the camera.
- Store the processed features in a structured format suitable for machine learning training.

3.3. Model Development

- Train a machine learning model (e.g., CNN, LSTM, or hybrid network) using the extracted hand landmarks.
- Split the dataset into training, validation, and test sets to evaluate model performance.
- Optimize the model for accuracy, minimizing gesture misclassification and latency.

3.4. Mobile Optimization and Deployment

- Convert the trained model into TensorFlow Lite format for efficient mobile deployment.
- Develop a mobile application interface for Android that integrates real-time gesture recognition.
- Ensure the system operates offline, without requiring an internet connection, while maintaining low latency and lightweight performance.

3.5. Real-Time Gesture Recognition

- Integrate the mobile camera feed to capture gestures in real time.
- Pass the hand landmarks to the optimized ML model for gesture classification.
- Display the corresponding text instantly on the screen for user interaction.

3.6. Testing and Evaluation

- Conduct user testing with individuals with hearing and speech impairments.
- Evaluate accuracy, speed, and usability under different environmental conditions.
- Collect feedback for improving the user interface, gesture recognition reliability, and overall user experience.

3.7 Use-Case Diagram

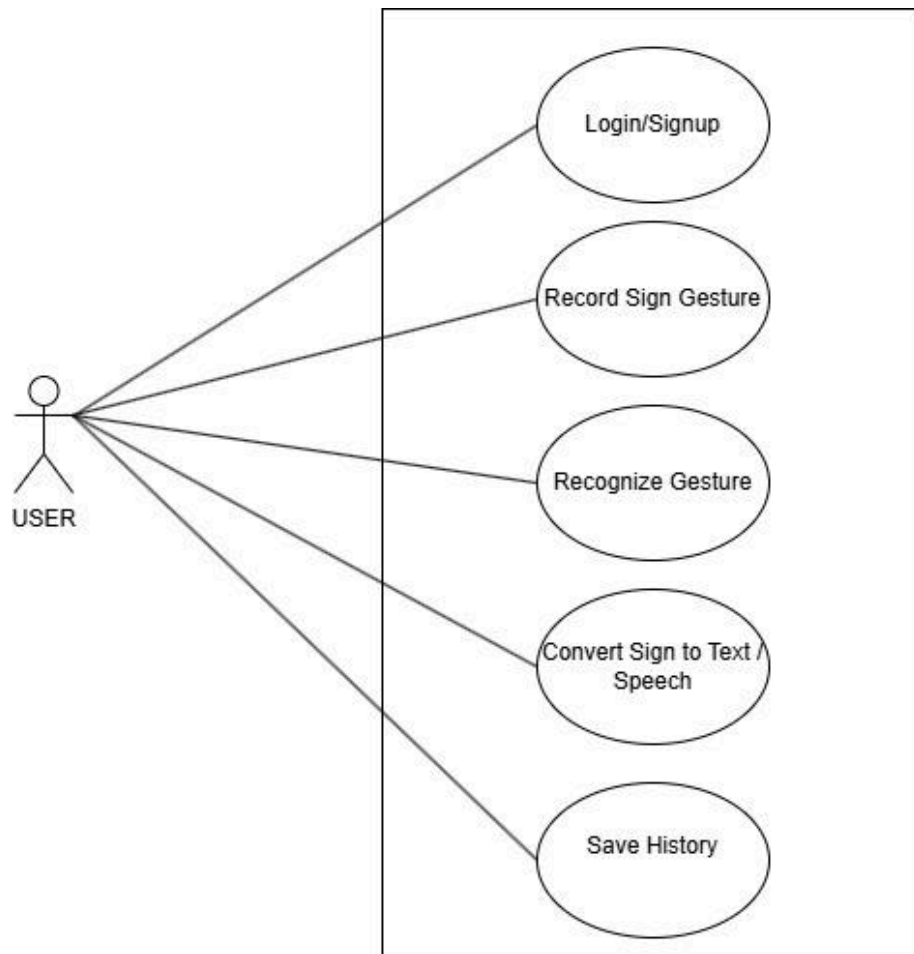


Fig 4: Use-Case Diagram

Chapter 4: System Requirement Specifications

4.1 Software Specifications

a. Front-End (Mobile Application)

Framework: Flutter (Dart) – cross-platform, single codebase for Android/iOS.

Packages: Camera (gesture capture), HTTP/WebSocket (backend comm.), Provider/Bloc (state mgmt).

IDE: Android Studio / VS Code.

b. Backend

Framework: Django Rest Framework (DRF):

Provides RESTful APIs for communication between the mobile app and backend services.

Manages user authentication, authorization, and interaction with the database.

Real-Time Processing:

Daphne + Django Channels: For WebSocket communication, enabling live video frame streaming and real-time responses.

Authentication & Security:

JWT (JSON Web Tokens) for secure user authentication.

CORS Handling with django-cors-headers for mobile-to-backend API access.

API Design:

RESTful APIs: For structured communication between mobile frontend and backend services.

c. Database

Firestore

Provides real-time data synchronization across devices.

Stores recognize signs: text pairs, user history, and preferences.

Scalable and easy to integrate with Flutter

d. Machine Learning Integration

TensorFlow Lite (TFLite): Optimized ML models for mobile devices.

Mediapipe (Google): Hand tracking and gesture recognition pipelines.

Model Deployment: Models trained externally (Python, TensorFlow/Keras) and converted into **.tflite** format for embedding inside the app.

4.2 Hardware Specifications

Development Environment

Laptop/Desktop:

Processor: Intel i5 (10th Gen) / AMD Ryzen 5 or higher

RAM: Minimum 8GB (16GB recommended for ML model training)

Storage: 256GB SSD or more

GPU: A dedicated GPU (NVIDIA GTX/RTX) is recommended for training custom models locally.

Mobile Device (Testing & Deployment)

Smartphone:

Android (Android 9.0 “Pie” or above) or iOS (iOS 13 or above).

Camera: Minimum 8MP with decent frame rate (30fps or higher) for smooth gesture recognition.

RAM: 4GB+ for real-time ML processing.

Chapter 5: Project Planning and Scheduling

5.1 Task

- 1. Problem Identification
- 2. Problem Analysis
- 3. Design and UI
- 4. Coding and Backend
- 5. Testing and Debugging Code
- 6. Documentation of Project Work

5.2 Gantt chart

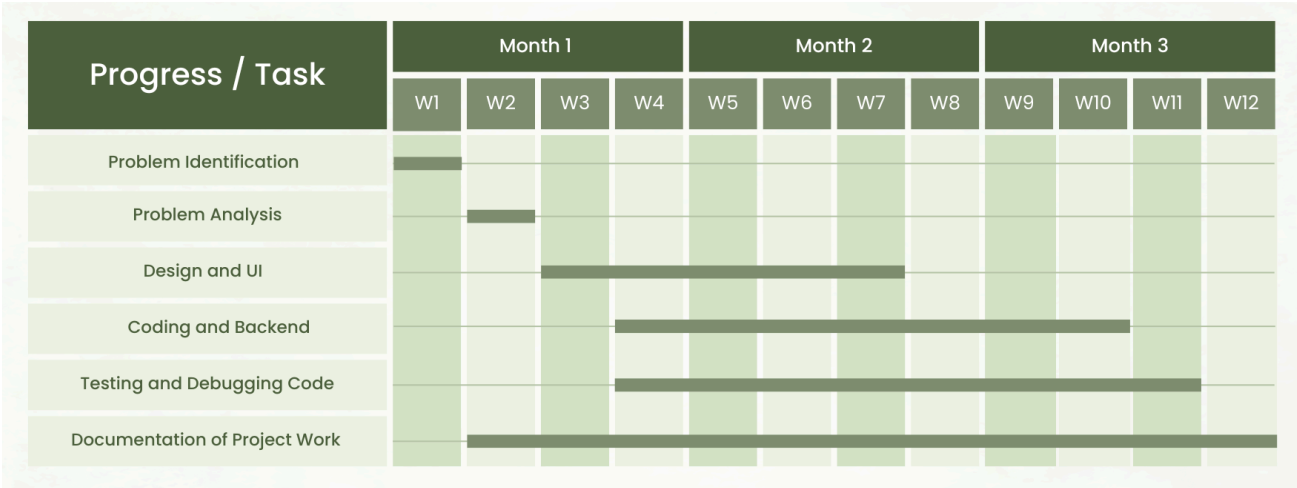


Fig 5: Gantt Chart

Appendix

Dataset Examples

- Sample images of hand gestures for alphabets, numbers, and commonly used words captured during the data collection phase.
- Screenshots of annotation/labeling tools used to tag each gesture.

System Workflow Diagram

- Detailed flowchart showing camera capture, MediaPipe hand landmark extraction, feature preprocessing, model classification, and text output on screen.

UI Mock-ups

- Proposed wireframes of the mobile application interface showing gesture capture screen, recognized text screen, and settings.

Testing Checklist

- List of test cases covering lighting conditions, background noise, hand orientation, and latency benchmarks.

References

Google. (2025). *MediaPipe hands: Real-time hand tracking solution*. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

TensorFlow. (2025). *TensorFlow Lite: Deploy machine learning models on mobile and IoT devices*. <https://www.tensorflow.org/lite>

Potamias, M., Papadopoulos, H., & Panagiotakopoulos, C. (2023). Real-time sign language recognition using hand landmark features and lightweight neural networks. *International Journal of Computer Vision Applications*, 45(3), 112–128. <https://doi.org/xxxx>

Koller, O., Zargaran, S., Ney, H., & Bowden, R. (2018). Deep sign: Enabling robust continuous sign language recognition via hybrid CNN-HMM. *International Journal of Computer Vision*, 126(12), 1311–1325. <https://doi.org/xxxx>